

MODULE 13 · DOMAIN 3 - DATA OPERATIONS AND SUPPORT

Querying and analysing

Amazon Athena, SQL that earns marks, Spark notebooks, and the BI service that changed its name twice.

DOMAIN 3 of 4 EXAM WEIGHT 22% MODULE 13 of 27

Concurrency and quotas

- ◆ **Active query quotas** apply per account and per Region, separately for DDL and DML.
- ◆ **Beyond the quota, queries queue.** Users see this as slowness, not as an error.
- ◆ **Capacity reservations give guaranteed concurrency** for a workgroup.
- ◆ **Separate workgroups** so one team's burst does not starve another's.
- ◆ Twenty workgroups can share a single reservation.

ON THE EXAM

Queue wait and execution time are different symptoms here too, exactly as they were in Redshift.

Athena questions are about **bytes scanned**. Almost nothing else moves the answer.

What Athena is, underneath

- ◆ **Serverless SQL** over data in Amazon S3. No cluster, no loading.
- ◆ **Built on Trino**, formerly PrestoSQL. Engine version three is current.
- ◆ **Schema from the AWS Glue Data Catalog**, shared with Redshift Spectrum and EMR.
- ◆ **Federated query** reaches other sources through connectors.
- ◆ **Nothing is stored in Athena**. It is a query engine and a workgroup, nothing more.

ON THE EXAM

Athena and Redshift Spectrum read the same catalog tables. One definition, two engines.

Workgroups, and what they control

- ◆ **Separate query environments** for teams or workloads in one account.
- ◆ **Per-query data scan limits** that cancel a query before it spends too much.
- ◆ **Per-workgroup limits** across a period, with alarms when they are approached.
- ◆ **A result location per workgroup**, and enforced encryption of results.
- ◆ **Cost allocation by workgroup**, so spend is attributable to a team.

ON THE EXAM

A scan limit is a guardrail against runaway spend. Raising it permits more cost; it never reduces any.

Athena engine versions and Iceberg

- ◆ **Engine version three** is current, built on a newer Trino release.
- ◆ **Better predicate pushdown** than version two, so some queries read less.
- ◆ **Set per workgroup**, or left on Auto to follow AWS defaults.
- ◆ **Full Iceberg support.** MERGE, UPDATE, DELETE and time travel in SQL.
- ◆ **CTAS for S3 Tables** since August 2025.

ON THE EXAM

Engine version three is what makes Athena a read-and-write engine over Iceberg rather than a read-only one.

Reducing what a query reads

- ◆ **Partition** so a filter skips whole prefixes. The largest single lever.
- ◆ **Columnar format** so only the requested columns are read.
- ◆ **Compress**, because you are billed for compressed bytes read.
- ◆ **Avoid SELECT star**. It defeats columnar storage entirely.
- ◆ **Query result reuse** returns a cached result for an identical query, free.
- ◆ **Small files** cost request overhead that never appears as bytes scanned.

ON THE EXAM

Together, partitioning and Parquet routinely cut a bill by ninety percent or more. Nothing else comes close.

What Athena actually charges for

CHARGE	NOTES
Data scanned	Five dollars per terabyte, rounded up per query to ten megabytes
Capacity reservations	Per DPU-hour instead. No data scanned charge at all
Athena for Spark	Per DPU-hour while the session is open
Federated query	Lambda charges for the connector, on top
Result storage	S3 storage for every query result, until a lifecycle rule expires it

ON THE EXAM

Failed and cancelled queries are still charged for what they scanned before stopping.

Pay per query, or reserve capacity?

ON DEMAND

Per terabyte scanned

- Five dollars per terabyte, nothing when idle
- Perfect for occasional and exploratory work
- Concurrency shared with the account
- Cost scales with how much you read

CAPACITY RESERVATIONS

Per DPU-hour

- Dedicated compute, no data scanned charges
- Predictable cost and guaranteed concurrency
- From four DPUs, billed by the minute
- Worth it at sustained heavy usage

THE TIEBREAKER

Reservations became far more practical in February 2026, dropping to a four DPU minimum with one minute billing.

A retailer's analysts query a two hundred terabyte clickstream dataset with Amazon Athena. The data is Parquet, partitioned by date, and queries filter on a date range. Monthly Athena spend has reached a level finance has flagged.

Investigation shows that most queries are written as SELECT star, and that four dashboards each re-run the same three queries every fifteen minutes against data that only changes once a day.

Which combination will reduce the bill most?

- ☐ A Increase the workgroup's per-query data scan limit so queries stop being cancelled
- ☐ B Select only the required columns, and enable query result reuse for the dashboard queries
- ☐ C Repartition the data by hour to improve pruning
- ☐ D Move the dataset to Amazon S3 Standard-Infrequent Access

◆ PAUSE AND DECIDE

Which combination will reduce the bill most?

- A

Raise the scan limit

A guardrail against overspend. Raising it permits more cost and reduces none, and nothing said queries were being cancelled.
- B

Named columns plus result reuse

SELECT star defeats Parquet entirely. Reuse makes repeated identical dashboard queries free against unchanged data.
- C

Repartition by hour

Queries filter by date range, which the daily partitions already serve. This adds partitions nobody filters on.
- D

Move to Standard-IA

Changes storage cost and adds a retrieval charge. Athena bills bytes scanned regardless of storage class.

THE SIGNAL

Two planted details, two halves of the answer. SELECT star and identical repeated queries.

The exam guide names **specific SQL constructs**. You will not write them, and you must reason about them.

The constructs the exam guide names

REQUIREMENT	CONSTRUCT
Totals per category	GROUP BY with an aggregate function
A running or rolling average	A window function with an OVER clause
Rank within a group	ROW_NUMBER or RANK, partitioned by the group
Rows into columns	PIVOT, or conditional aggregation with CASE
Reusable intermediate results	A common table expression, using WITH
Compare to the previous period	LAG or LEAD over an ordered window

ON THE EXAM

Window functions are the family to be fluent in. Rolling averages, rankings and period comparisons all use them.

GROUP BY or a window function?

GROUP BY

Collapse the rows

- One output row per group
- Detail is lost
- Totals, counts, averages per category
- Cheaper, because there is less output

WINDOW FUNCTION

Keep every row

- Every input row survives, with a computed column added
- Rolling averages, running totals, rankings
- Compares a row to its neighbours
- More expensive, because nothing is collapsed

THE TIEBREAKER

If the requirement says "alongside", "running", "rolling" or "rank within", it is a window function.

Querying nested data

- ◆ **JSON and Parquet both carry nested structures**, arrays and objects inside a column.
- ◆ **Dot notation** reaches into a struct. Customer dot address dot city.
- ◆ **UNNEST turns an array into rows**, one per element, so it can be aggregated.
- ◆ **Query nested data in place** rather than flattening the whole dataset first.
- ◆ **Glue relationalize** is the alternative, flattening into related tables at load time.

ON THE EXAM

Two answers to nested data. Query it where it sits, or flatten it on the way in. The exam offers both.

Making a Trino query faster

- ◆ **Reduce before you join.** Filter and aggregate each side first.
- ◆ **Put the larger table on the left** of a join, by convention in Trino.
- ◆ **Approximate functions** like `approx_distinct` are far cheaper than exact counts.
- ◆ **ORDER BY across a huge result** funnels everything through one worker.
- ◆ **LIMIT with ORDER BY** lets the engine avoid a full sort.

ON THE EXAM

An exact distinct count over billions of rows is expensive. An approximate one is usually good enough and much cheaper.

Getting results out of Athena

- ◆ **Every query writes its result** to the workgroup's S3 location automatically.
- ◆ **UNLOAD** writes results in Parquet, ORC, Avro or JSON, in parallel.
- ◆ **CTAS** creates a table and its data in one statement, as module five covered.
- ◆ **Athena's API is asynchronous.** Start a query, poll, then read the result location.
- ◆ **Pulling millions of rows to a client** is slow and usually the wrong design.

ON THE EXAM

UNLOAD in Parquet is how a query result becomes a dataset other engines can read.

Views, and reusable logic

- ◆ A **view** stores a query definition, not results. It runs every time.
- ◆ **Good for hiding complexity** and for restricting which columns consumers see.
- ◆ **It saves no scanning**, because the underlying query still executes in full.
- ◆ **Athena materialized views** do store results and refresh on a schedule.
- ◆ **Redshift materialized views** are the equivalent inside the warehouse.

ON THE EXAM

A view simplifies. A materialized view accelerates. Questions about cost want the second one.

Why a query is slow or expensive

- ◆ **Filter early.** A predicate applied before a join reduces what the join handles.
- ◆ **Project narrowly.** Name the columns you need, and no others.
- ◆ **Joining large to large** is the expensive shape. Reduce one side first.
- ◆ **Filter on the partition column,** not on a value derived from it.
- ◆ **ORDER BY on a huge result** forces everything through one node.

ON THE EXAM

The fourth point is the silent one. A filter on a derived value looks correct and prunes nothing.

Where the SQL should run

- ◆ **Athena** for occasional and exploratory queries over the lake.
- ◆ **Redshift** for repeated, concurrent queries over curated data.
- ◆ **Redshift Spectrum** to join warehouse tables to lake data in one statement.
- ◆ **Athena federated query** to reach an operational source occasionally.
- ◆ The same SQL can run in several of these. Frequency decides, not capability.

ON THE EXAM

Module one's rule again. Occasional and unpredictable favours Athena. Constant and concurrent favours Redshift.

What federated query can reach

SOURCE	NOTES
Amazon DynamoDB	Join operational items to lake data. Consumes read capacity
Amazon RDS and Aurora	Live relational data. Puts load on the operational database
Amazon DocumentDB, Redshift, OpenSearch	Native connectors
On-premises and third-party	Custom connectors, running as Lambda functions

ON THE EXAM

Every connector runs as a Lambda function you pay for, and every query reaches into a live system.

An analyst needs a report showing, for each product, daily sales alongside a seven day rolling average of those sales, and the rank of each product within its category by monthly revenue. The data is three years of transactions in Amazon Redshift.

A colleague has proposed exporting the data to Amazon S3, computing the rolling average and rankings in an AWS Glue job with Python, and loading the results back into Redshift.

What should the data engineer recommend?

- ☐ A Build the Glue job as proposed, since window calculations are easier in Python
- ☐ B Compute both with SQL window functions directly in Redshift
- ☐ C Export to Amazon S3 and use Amazon Athena, which handles window functions better than Redshift
- ☐ D Load the data into Amazon QuickSight and use calculated fields

◆ PAUSE AND DECIDE

What should the data engineer recommend?

- A

Export, compute in Glue, load back

Data out of the warehouse and back in to do work the warehouse does natively. Module five marked this shape wrong.
- B

Window functions in Redshift

A rolling average and a partitioned rank are exactly what window functions are for, and the data is already there.
- C

Export to Athena

Both engines support window functions equally. This adds an export to reach a capability Redshift already has.
- D

Calculated fields in the BI tool

Pushes the computation to the visualisation layer for every viewer, rather than computing it once.

THE SIGNAL

"Rolling average" and "rank within a category" both name window functions. The data is already in the warehouse.

When SQL is not the right language, Athena runs **Spark notebooks** too.

Athena SQL or Athena for Spark?

ATHENA SQL

Declarative

- Ad hoc and scheduled queries
- Billed per terabyte scanned
- Anyone who knows SQL can use it
- Result is a table

ATHENA FOR APACHE SPARK

Programmatic

- Notebooks, in Python or Scala
- Billed per DPU-hour while the session is open
- Libraries, iteration, complex reshaping
- Result is whatever you write

THE TIEBREAKER

An idle notebook session bills for as long as it stays open. That is the cost trap, exactly as in module five.

Amazon QuickSight has been renamed **twice**. Both old and new names appear in current material.

What the rename means for the exam

- ◆ QuickSight became Amazon Quick Suite on 9 October 2025.
- ◆ Quick Suite became Amazon Quick during 2026.
- ◆ The BI component inside it is called Quick Sight, with a space.
- ◆ The in-scope service list says Amazon Quick. Older questions will say QuickSight.
- ◆ Treat all three names as the same answer. The capability has not moved.

ON THE EXAM

Availability warning: recognise QuickSight, Quick Suite and Quick as the same service. Nothing about the BI capability changed.

The BI capability, whatever it is called

- ◆ **Dashboards and visualisations** over Redshift, Athena, RDS, S3 and more.
- ◆ **SPICE** is the in-memory engine that caches data for fast dashboards.
- ◆ **Query data directly** instead, when freshness matters more than speed.
- ◆ **Authors and readers** are licensed separately, which shapes cost questions.
- ◆ **Embedding** puts dashboards inside another application.

ON THE EXAM

SPICE is the concept to know. It decides whether a dashboard hits the warehouse on every view or reads a cached copy.

What the rebrand actually added

- ◆ **Amazon Quick is a wider product** than QuickSight was, not just a new name.
- ◆ **Agentic components** for research, automation and indexing sit alongside the BI tool.
- ◆ **Natural language questions** over connected data, grounded in defined spaces.
- ◆ **The BI engine and SPICE are unchanged.** Dashboards behave exactly as before.
- ◆ Existing APIs, SDKs and embedded dashboards continue to work.

ON THE EXAM

For a data engineering exam, the BI capability is what matters. The agentic layer is context, not content.

A company publishes an executive dashboard built on Amazon Redshift. It is opened by about six hundred people each weekday morning, mostly within the same half hour. The underlying data is refreshed once overnight and does not change during the day.

During that half hour, the Redshift cluster shows heavy query load and other analytical workloads slow noticeably. The dashboard queries themselves are identical for every viewer.

What is the most appropriate change?

- ☐ A Import the dashboard dataset into SPICE and refresh it after the overnight load
- ☐ B Enable Redshift concurrency scaling so the extra queries run on additional clusters
- ☐ C Resize the Redshift cluster to handle the morning peak
- ☐ D Move the dashboard to query Amazon Athena over the same data in Amazon S3

◆ PAUSE AND DECIDE

What is the most appropriate change?

- A

 Import into SPICE, refresh after the load
Six hundred identical queries against static data become zero. The dashboard reads a cached copy and the cluster is untouched.
- B

 Concurrency scaling
Absorbs the burst by adding clusters, and pays to run six hundred identical queries that did not need running.
- C

 Resize the cluster
Permanent capacity for a half hour event, and the redundant queries still run.
- D

 Point the dashboard at Athena
Moves the load rather than removing it. Six hundred scans of the same data, now billed per terabyte.

THE SIGNAL

Identical queries against data that does not change is the caching signal. Removing work beats absorbing it.

Preparing data without writing code

- ◆ **AWS Glue DataBrew** offers over two hundred and fifty prebuilt transformations.
- ◆ **Glue Studio data preparation** is the successor path, and imports DataBrew recipes.
- ◆ **Profiling** reports distributions, missing values and outliers before you transform.
- ◆ **Recipes are reusable**, so an exploration becomes a repeatable job.
- ◆ Aimed at analysts, not engineers. That framing is what questions test.

ON THE EXAM

The exam guide names both DataBrew and SageMaker Unified Studio for data preparation. Either can be correct.

Making a dashboard fast

- ◆ **Aggregate before the BI tool sees it.** A dashboard should read summaries, not raw rows.
- ◆ **Materialized views in Redshift** precompute the aggregation once.
- ◆ **SPICE** caches the result set in memory for the viewers.
- ◆ **Calculated fields in the BI layer** recompute for every viewer, every time.
- ◆ **Push filters down** so the source does the reduction, not the dashboard.

ON THE EXAM

Compute once, read many times. Every item here is a version of that principle.

Keeping a dashboard current

- ◆ **Scheduled SPICE refresh** after the upstream pipeline completes, not on a fixed clock.
- ◆ **Incremental refresh** loads only new rows, for large datasets.
- ◆ **Refreshing on a clock** can import data mid-load, showing partial figures.
- ◆ **Threshold alerts** notify viewers when a value crosses a boundary.
- ◆ **Anomaly detection** highlights unexpected changes without anyone defining a rule.

ON THE EXAM

Refresh after the pipeline signals completion. A clock-based refresh eventually catches a load in progress.

Getting a dashboard to its audience

- ◆ **Readers** view and interact. **Authors** build. They are licensed differently.
- ◆ **Embedding** puts a dashboard inside your own application or portal.
- ◆ **Anonymous embedding** serves users who have no account at all.
- ◆ **Scheduled email reports** for people who will never open a dashboard.
- ◆ **Per-viewer pricing** makes wide distribution a cost question, not just a technical one.

ON THE EXAM

Six hundred readers and six authors is a very different bill from six hundred authors. Questions state the split.

Where to compute an aggregation

LAYER	CHARACTER
In the source query	Always current, recomputed on every request
Materialized view	Computed once per refresh, read by any consumer
SPICE import	Cached in memory, refreshed on a schedule, isolates the source
BI calculated field	Recomputed per viewer. Flexible and the most expensive

ON THE EXAM

Freshness decreases and efficiency increases as you move down this table. That is the trade to reason about.

Governing who sees what

- ◆ **Row-level security** in the BI tool restricts rows per user or group.
- ◆ **Column-level security** hides columns from unauthorised viewers.
- ◆ **Lake Formation permissions** apply when the query reaches through the catalog.
- ◆ **SPICE caches results**, so security must be applied to the imported dataset too.
- ◆ Module nineteen covers masking and encryption in full.

ON THE EXAM

An import into SPICE is a copy. Access rules on the source do not follow the copy unless you configure them.

A data science team needs to explore a new one hundred gigabyte dataset in Amazon S3. The work is iterative and involves reshaping nested structures, testing several statistical approaches, and using a Python library that has no SQL equivalent.

The team runs this kind of exploration perhaps twice a month, for a day at a time. They have no existing Amazon EMR clusters and no capacity to operate one.

What should the data engineer recommend?

- ☐ A Provision a long-running Amazon EMR cluster with EMR Studio notebooks
- ☐ B Use Athena for Apache Spark notebooks, shutting the session when work finishes
- ☐ C Write the analysis as a series of Athena SQL queries
- ☐ D Load the dataset into Amazon Redshift and analyse it with SQL

◆ PAUSE AND DECIDE

What should the data engineer recommend?

- A

 A long-running EMR cluster
A cluster running continuously for two days of work a month, from a team who said they cannot operate one.
- B

 Athena for Apache Spark notebooks
Serverless Spark with libraries, over the same data and catalog, billed only while the session is open.
- C

 A series of Athena SQL queries
The requirement names a Python library with no SQL equivalent. That rules SQL out explicitly.
- D

 Load into Redshift and use SQL
Same objection, plus loading a hundred gigabytes into a warehouse for two days of exploration a month.

THE SIGNAL

"A Python library with no SQL equivalent" eliminates both SQL options before you weigh anything else.

Where to put a visualisation

SITUATION	ANSWER
Business dashboards for many viewers	Amazon Quick, formerly QuickSight
Exploring data before transforming it	Glue DataBrew profiling
Log and operational dashboards	Amazon OpenSearch Service dashboards
Data science notebooks and charts	Athena for Apache Spark, or SageMaker
Infrastructure metrics	Amazon Managed Grafana, or CloudWatch dashboards

ON THE EXAM

Log dashboards go to OpenSearch, not to the BI tool. That split appears in questions.

Symptoms and their causes

SYMPTOM	LIKELY CAUSE
Athena bill high on a partitioned Parquet table	SELECT star, or filters on a derived value
The same query costs the same every time	Query result reuse not enabled
A dashboard slows the warehouse each morning	Direct query where SPICE would isolate it
A query returns correct results and scans everything	The filter does not match the partition column
Notebook charges continue overnight	A session left open. Billing is by session, not by query
A rolling average computed in a Glue job	A window function would do it in the warehouse

ON THE EXAM

The fourth row is the silent one. Correct answers, full table scans, and nothing reports a problem.

Reading a query and analysis question quickly

- ◆ **Is the cost bytes scanned?** Then only reading less will help.
- ◆ **Is the work redundant?** Caching beats capacity when queries repeat identically.
- ◆ **Occasional or constant?** Athena and Redshift split on frequency, not capability.
- ◆ **Does it need a language SQL cannot express?** That is a notebook.
- ◆ **Where should the aggregation happen?** Once in the source, not per viewer.

ON THE EXAM

Next: monitoring, logging and troubleshooting. The largest module in Domain 3.